Java Design Principles

SOLID, DRY, and KISS - A Student-Friendly Guide with Code Examples

Why do we use design principles? They help us write code that is easy to understand, easy to change, and easy to reuse. Bad code breaks when requirements change. Good code adapts.

This guide explains each principle in plain English with simple Java examples you can run.

SOLID Principles

SOLID is an acronym for 5 design principles that make software more maintainable and flexible.

S Single Responsibility Principle (SRP)

1. SIMPLE MEANING

> A class should have only one reason to change. Each class should do just one job.

2. WHY IT MATTERS

> If a class does too many things, changing one part can break another unrelated part. It's harder to test and reuse.

3. BAD VS GOOD CONCEPT

✗ Bad: One class does everything

```java
class Employee {
    private String name;
    private double salary;

    // Job 1: Calculate pay
    public double calculatePay() {
        return salary / 12;
    }

    // Job 2: Save to database
    public void saveToDatabase() {
        // DB logic here
    }

    // Job 3: Generate report
    public String generateReport() {
        return "Report for " + name;
    }
}
```

✓ Good: Each class has one job

```java
// Job 1: Only data
class Employee {
    private String name;
    private double salary;

    public double getMonthlyPay() {
        return salary / 12;
    }
    // getters/setters...
}

// Job 2: Only database
class EmployeeRepository {
    public void save(Employee e) {
        // DB logic here
    }
}

// Job 3: Only reports
class EmployeeReportGenerator {
    public String generate(Employee e) {
        return "Report for " + e.getName();
    }
}
```

O Open/Closed Principle (OCP)

1. SIMPLE MEANING

> Software should be open for extension but closed for modification. You should add new features by adding new code, not by changing existing code.

2. WHY IT MATTERS

> Changing existing, working code risks breaking things. If you can extend with new classes instead, your system is safer.

3. BAD VS GOOD CONCEPT

✗ Bad: Modify for each new payment

```java
class PaymentProcessor {
    public void process(String type) {
        if (type.equals("CREDIT")) {
            // process credit card
        } else if (type.equals("PAYPAL")) {
            // process paypal
        } else if (type.equals("CRYPTO")) {
            // need to modify this class!
        }
    }
}
```

✓ Good: Extend with new class

```java
interface PaymentMethod {
    void processPayment(double amount);
}

class CreditCardPayment implements PaymentMethod
    @Override
    public void processPayment(double amount) {
        // process credit card
    }
}

class PayPalPayment implements PaymentMethod {
    @Override
    public void processPayment(double amount) {
        // process paypal
    }
}

// Add crypto? Just create new class.
// No existing code changes!
class CryptoPayment implements PaymentMethod {
    @Override
    public void processPayment(double amount) {
        // process crypto
    }
}
```

L Liskov Substitution Principle (LSP)

1. SIMPLE MEANING

> If class B is a subclass of class A, you should be able to use B anywhere you use A without breaking anything. Subclasses should not break parent behavior.

2. WHY IT MATTERS

> If a subclass changes the expected behavior, your code becomes unpredictable and full of bugs.

3. BAD VS GOOD CONCEPT

✗ Bad: Penguin can't fly but Bird says it can

```java
class Bird {
    public void fly() {
        // fly in the sky
    }
}

class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException(
            "Penguins can't fly!"
        );
    }
}

// This breaks! We expect all Birds to fly
Bird bird = new Penguin();
bird.fly(); // CRASH
```

✓ Good: Only flying birds can fly

```java
class Bird {
    // Common bird behavior
}

class FlyingBird extends Bird {
    public void fly() {
        // fly in the sky
    }
}

class Eagle extends FlyingBird {
    // Eagle can fly, no surprises
}

class Penguin extends Bird {
    public void swim() {
        // Penguins swim instead
    }
}
// Now we only call fly() on FlyingBird
```

I Interface Segregation Principle (ISP)

1. SIMPLE MEANING

> Don't force classes to implement methods they don't use. Make small, specific interfaces instead of one big interface.

2. WHY IT MATTERS

> Fat interfaces create useless code and dependencies. Classes end up with empty methods or throwing exceptions.

3. BAD VS GOOD CONCEPT

✗ Bad: Fat interface forces everyone to eat

```java
interface Worker {
    void work();
    void eat();
}

// Human can do both, fine
class Human implements Worker {
    public void work() { /*...*/ }
    public void eat() { /*...*/ }
}

// Robot doesn't eat! Forced to implement.
class Robot implements Worker {
    public void work() { /*...*/ }
    public void eat() {
        throw new UnsupportedOperationException()
    }
}
```

✓ Good: Split into small interfaces

```java
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

// Human implements both
class Human implements Workable, Eatable {
    public void work() { /*...*/ }
    public void eat() { /*...*/ }
}

// Robot only implements what it needs
class Robot implements Workable {
    public void work() { /*...*/ }
    // No useless eat() method!
}
```

D Dependency Inversion Principle (DIP)

1. SIMPLE MEANING

> High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces). Depend on interfaces, not concrete classes.

2. WHY IT MATTERS

> If your high-level business logic depends on specific low-level details, you can't change details without rewriting business logic. It makes testing hard too.

3. BAD VS GOOD CONCEPT

✗ Bad: Depends on concrete class

```java
class LightBulb {
    public void turnOn() { /*...*/ }
    public void turnOff() { /*...*/ }
}

class Switch {
    private LightBulb bulb = new LightBulb();

    public void operate() {
        bulb.turnOn(); // Tied to LightBulb!
    }
}
// What if we want to control a Fan?
// We must change Switch class.
```

✓ Good: Depends on interface

```java
// Abstraction
interface Switchable {
    void turnOn();
    void turnOff();
}

class LightBulb implements Switchable {
    public void turnOn() { /*...*/ }
    public void turnOff() { /*...*/ }
}

class Fan implements Switchable {
    public void turnOn() { /*...*/ }
    public void turnOff() { /*...*/ }
}

class Switch {
    private Switchable device;

    public Switch(Switchable device) {
        this.device = device; // Inject!
    }

    public void operate() {
        device.turnOn(); // Works with any!
    }
}
```

Other Core Principles

DRY Don't Repeat Yourself

1. SIMPLE MEANING

> Every piece of knowledge must have a single, unambiguous representation in the system. Don't copy-paste code. Put it in one place and reuse it.

2. WHY IT MATTERS

> If you copy code and later need to change it, you must remember to change it everywhere. You'll forget, and bugs will happen. One source of truth is easier to maintain.

3. BAD VS GOOD CONCEPT

✗ Bad: Same logic repeated

```java
class Payroll {
    public double calculateBonus(Manager m) {
        // 10% bonus logic
        return m.getSalary() * 0.1;
    }

    public double calculateBonus(Developer d) {
        // Same 10% bonus logic copied!
        return d.getSalary() * 0.1;
    }
}
// If bonus changes to 12%,
// we must update in 2 places
```

✓ Good: One method for all

```java
interface Employee {
    double getSalary();
}

class Manager implements Employee { /*...*/ }
class Developer implements Employee { /*...*/ }

class PayrollService {
    private static final double BONUS_RATE = 0.1;

    // One method works for any Employee
    public double calculateBonus(Employee e) {
        return e.getSalary() * BONUS_RATE;
    }
}
// Change BONUS_RATE once,
// it works everywhere
```

KISS Keep It Simple, Stupid

1. SIMPLE MEANING

> Most systems work best if they are kept simple rather than made complicated. Don't over-engineer. Use the simplest solution that works.

2. WHY IT MATTERS

> Complex code is hard to read, hard to debug, and hard for others to understand. Simple code has fewer bugs and is easier to change later.

3. BAD VS GOOD CONCEPT

✗ Bad: Overcomplicated addition

```java
public class AdditionEngine {
    public int add(int a, int b) {
        List<Integer> numbers =
            new ArrayList<>();
        numbers.add(a);
        numbers.add(b);

        return numbers.stream()
            .mapToInt(Integer::intValue)
            .reduce(0, (x, y) -> x + y);
    }
}
// Way too complex for a + b
```

✓ Good: Simple and clear

```java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
// Does exactly what it says.
// Easy to read. Easy to test.
// KISS: Keep It Simple!
```

Quick Summary Table

PRINCIPLE	CORE IDEA	PROBLEM IT SOLVES	RULE OF THUMB
SRP	One class = One job	Classes that do too much break easily	If you use "and" to describe your class, split it
OCP	Add code, don't change code	Modifying working code creates bugs	Use interfaces/abstract classes for things that change
LSP	Subclasses must behave like parents	Surprising behavior from inheritance	If it looks like a duck and quacks, it should swim too
ISP	Many small interfaces > one big interface	Classes forced to implement useless methods	Clients shouldn't depend on methods they don't use
DIP	Depend on abstractions, not concretions	Tight coupling makes code rigid	High-level code should not import low-level code
DRY	Don't repeat yourself	Copy-paste bugs and maintenance nightmare	If you copy code twice, make a method
KISS	Keep it simple	Over-engineering creates complexity	Solve the problem you have, not the one you imagine